

1. Estime os parâmetros β_0, β_1 por GMM utilizando o estimador apresentado acima. Escreva um programa em Matlab utilizando o solver fminsearch.

```
In [1]: import pandas as pd

chilean = pd.read_csv('chilean.csv')

In [2]: import numpy as np
import pandas as pd
import statsmodels.api as sm
from scipy.optimize import minimize

df = chilean.rename(columns={'Y': 'y', 'sX': 'k', 'pX': 'm', 'fx1': 'fx1', 'fx2': 'fx2'}).copy()

# L = ln(exp(fx1) + exp(fx2))
df['l'] = np.log(np.exp(df['fx1']) + np.exp(df['fx2']))

# Selecionar variáveis relevantes e criar uma cópia explícita
df_panel = df[['idvar', 'timevar', 'y', 'k', 'l', 'm']].copy()

# Ordenar dados para Lagging
df_panel.sort_values(['idvar', 'timevar'], inplace=True)

# Primeiro Estágio: OLS

# Criar termos polinomiais de 4ª ordem para f^{l}(k, l, m)
X_stage1 = pd.DataFrame(index=df_panel.index)
X_stage1['const'] = 1
X_stage1['k'] = df_panel['k']
X_stage1['l'] = df_panel['l']

# $sum_{i=0}^4 \beta_i k^i * L^{4-i}$
X_stage1['l_4'] = df_panel['l']**4
X_stage1['k_l_3'] = df_panel['k'] * (df_panel['l']**3)
X_stage1['k2_l2'] = (df_panel['k']**2) * (df_panel['l']**2)
X_stage1['k3_l'] = (df_panel['k']**3) * df_panel['l']
X_stage1['k_4'] = df_panel['k']**4

# m^4 + k^4m^3 + k^2m^2 + k^3m (k^4 já foi definido)
X_stage1['m_4'] = df_panel['m']**4
X_stage1['k_m_3'] = df_panel['k'] * (df_panel['m']**3)
X_stage1['k2_m2'] = (df_panel['k']**2) * (df_panel['m']**2)
X_stage1['k3_m'] = (df_panel['k']**3) * df_panel['m']

# m^3L^3 + m^2L^2 + m^3L (L^4 e m^4 já foram definidos)
X_stage1['m_l_3'] = df_panel['m'] * (df_panel['l']**3)
X_stage1['m2_l2'] = (df_panel['m']**2) * (df_panel['l']**2)
X_stage1['m3_l'] = (df_panel['m']**3) * df_panel['l']

y_stage1 = df_panel['y']

data_stage1_full = pd.concat([y_stage1, X_stage1], axis=1)
data_stage1_clean = data_stage1_full.dropna()

y_stage1_clean = data_stage1_clean['y']
X_stage1_clean = data_stage1_clean.drop(columns=['y'])

model_stage1 = sm.OLS(y_stage1_clean, X_stage1_clean)
results_stage1 = model_stage1.fit()
print(results_stage1.summary())

beta_0_tilde = results_stage1.params['const']
print(f"Intercepto do primeiro estágio: {beta_0_tilde}")

# Calcular Phi_hat e adicionar de volta ao df_panel original usando o índice
df_panel['Phi_hat'] = np.nan
df_panel.loc[X_stage1_clean.index, 'Phi_hat'] = results_stage1.predict(X_stage1_clean)

=====
OLS Regression Results
=====
Dep. Variable: y R-squared: 0.824
Model: OLS Adj. R-squared: 0.823
Method: Least Squares F-statistic: 843.9
Date: Fri, 20 Jun 2025 Prob (F-statistic): 0.00
Time: 16:57:44 Log-likelihood: -2369.4
No. Observations: 2544 AIC: 4769.
DF Residuals: 2529 BIC: 4856.
DF Model: 14
Covariance Type: nonrobust
=====
coef std err t P>|t| [0.025 0.975]
-----
const 9.2094 0.140 65.592 0.000 8.934 9.485
k 0.0015 0.019 0.078 0.937 -0.036 0.039
l 1.0511 0.047 22.236 0.000 0.958 1.144
l_4 -0.0132 0.005 -2.724 0.006 -0.023 -0.004
k_l_3 0.0095 0.005 1.788 0.074 -0.001 0.020
k2_l2 -0.0012 0.002 -0.639 0.523 -0.005 0.003
k3_l -0.0006 0.000 -2.216 0.027 -0.001 -7.06e-05
k_4 0.0002 1.94e-05 8.620 0.000 0.000 0.000
m_4 0.0055 0.001 6.179 0.000 0.004 0.007
k_m_3 -0.0097 0.001 -7.630 0.000 -0.012 -0.007
k2_m2 0.0049 0.001 7.245 0.000 0.004 0.006
k3_m -0.0008 0.000 -5.480 0.000 -0.001 -0.001
m_l_3 0.0092 0.003 3.152 0.002 0.003 0.015
m2_l2 -0.0160 0.004 -4.533 0.000 -0.023 -0.009
m3_l 0.0082 0.002 4.881 0.000 0.005 0.012
=====
Omnibus: 79.225 Durbin-Watson: 0.700
Prob(Omnibus): 0.000 Jarque-Bera (JB): 169.446
Skew: 0.178 Prob(JB): 1.60e-37
Kurtosis: 4.213 Cond. No. 3.52e+05
=====

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 3.52e+05. This might indicate that there are strong multicollinearity or other numerical problems.

Intercepto do primeiro estágio: 9.209416673947127
```

```
In [3]: # Segundo Estágio: GMM

# Criar variáveis defasadas (k_{t-1}, L_{t-1}, Phi_hat_{t-1})
df_panel['k_lag1'] = df_panel.groupby('idvar')['k'].shift(1)
df_panel['l_lag1'] = df_panel.groupby('idvar')['l'].shift(1)
df_panel['Phi_hat_lag1'] = df_panel.groupby('idvar')['Phi_hat'].shift(1)

# Selecionar apenas as colunas necessárias para o GMM
df_stage2 = df_panel[['y', 'k', 'l', 'Phi_hat_lag1', 'k_lag1', 'l_lag1']].dropna()

y_jt = df_stage2['y'].values
k_jt = df_stage2['k'].values
l_jt = df_stage2['l'].values
Phi_hat_t_minus_1 = df_stage2['Phi_hat_lag1'].values
k_jt_minus_1 = df_stage2['k_lag1'].values
l_jt_minus_1 = df_stage2['l_lag1'].values

rho_hat = 0.9528 # Valor fixo conforme a questão

# Função de momentos para GMM
def gmm_moments(params_gmm):
    beta_k, beta_l = params_gmm

    # Resíduo u_jt da Equação (6)
    # u_jt = y_jt - beta_0_tilde - beta_k*k_jt - beta_l*L_jt - rho_hat * (Phi_hat_{t-1} - beta_0_tilde - beta_k*k_{t-1} - beta_l*L_{t-1})

    term_current_prod = y_jt - beta_0_tilde - beta_k * k_jt - beta_l * l_jt
    term_lagged_prod_expectation = Phi_hat_t_minus_1 - beta_0_tilde - beta_k * k_jt_minus_1 - beta_l * l_jt_minus_1

    u_jt = term_current_prod - rho_hat * term_lagged_prod_expectation

    # Condições de momento E[u_jt * Z_jt] = 0
    # Z_jt = [k_jt, l_jt_minus_1]
    moment1 = np.mean(u_jt * k_jt) # Instrumento: k_jt
    moment2 = np.mean(u_jt * l_jt_minus_1) # Instrumento: l_jt-1

    return np.array([moment1, moment2])

# Função objetivo para GMM (soma dos quadrados dos momentos)
# Para GMM exatamente identificado, a matriz de ponderação W pode ser a identidade.
def gmm_objective(params_gmm):
    moments = gmm_moments(params_gmm)
    return np.sum(moments**2) # Equivalente a m'I m

# Valores iniciais para beta_k, beta_l (podem vir de um OLS simples ou serem arbitrários razoáveis)
# Usando os coeficientes de k e l do primeiro estágio como ponto de partida.
# Valores iniciais comuns para elasticidades.
initial_params_gmm = [0.1, 0.6]

print("\nSegundo Estágio - GMM:")
# Otimização usando Nelder-Mead (análogo ao fminsearch do Matlab)
result_gmm = minimize(gmm_objective, initial_params_gmm, method='Nelder-Mead')

print("Optimization successful.")
beta_k_gmm, beta_l_gmm = result_gmm.x
print(f"Final GMM objective function value: {result_gmm.fun:.6e}")
print(f"Estimated beta_k: {beta_k_gmm:.4f}")
print(f"Estimated beta_l: {beta_l_gmm:.4f}")

Segundo Estágio - GMM:
Optimization successful.
Final GMM objective function value: 6.993736e-12
Estimated beta_k: 0.3618
Estimated beta_l: -0.1739
```

2. Calcule a estatística de Wald (F)

```
In [4]: from scipy.stats import f as f_dist
import numpy as np

N_gmm = len(y_jt) # Número de observações na estimação GMM
K_params = 2 # Número de parâmetros estimados (beta_k, beta_l)
q_restrictions = 2 # Número de restrições (beta_k=0, beta_l=0)

# 1. Calcular as derivadas parciais de u_jt em relação a beta_k e beta_l
u_jt = y_jt - beta_0_tilde - beta_k*k_jt - beta_l*L_jt - rho_hat * (Phi_hat_t_minus_1 - beta_0_tilde - beta_k*k_jt_minus_1 - beta_l*L_jt_minus_1)
# d(u_jt)/d(beta_k) = -k_jt + rho_hat * k_jt_minus_1
# d(u_jt)/d(beta_l) = -L_jt + rho_hat * l_jt_minus_1

du_dbk = -k_jt + rho_hat * k_jt_minus_1
du_dbl = -l_jt + rho_hat * l_jt_minus_1

# 2. Calcular G_hat (Jacobiano médio das condições de momento)
# Instrumentos Z_jt = [k_jt, l_jt_minus_1]
# G_hat é uma matriz 2x2, onde G_hat[i,j] = mean( Z_i * d(u_jt)/d(beta_j) )
# G_hat = E[Z_jt * (du/d_beta)]
# G_hat_row1 = [mean(k_jt * du_dbk), mean(k_jt * du_dbl)]
# G_hat_row2 = [mean(l_jt_minus_1 * du_dbk), mean(l_jt_minus_1 * du_dbl)]
# No entanto, a fórmula padrão para V_cov usa G = E[ (du/d_beta) * Z' ]
# G_hat_col1 = [mean(du_dbk * k_jt), mean(du_dbk * l_jt_minus_1)]
# G_hat_col2 = [mean(du_dbl * k_jt), mean(du_dbl * l_jt_minus_1)]

G_hat_11 = np.mean(du_dbk * k_jt)
G_hat_12 = np.mean(du_dbl * k_jt)
G_hat_21 = np.mean(du_dbk * l_jt_minus_1)
G_hat_22 = np.mean(du_dbl * l_jt_minus_1)
G_hat = np.array([[G_hat_11, G_hat_12], [G_hat_21, G_hat_22]])

# 3. Calcular resíduos u_jt_hat com os parâmetros GMM estimados
term_current_prod_hat = y_jt - beta_0_tilde - beta_k_gmm * k_jt - beta_l_gmm * l_jt
term_lagged_prod_expectation_hat = Phi_hat_t_minus_1 - beta_0_tilde - beta_k_gmm * k_jt_minus_1 - beta_l_gmm * l_jt_minus_1
u_jt_hat = term_current_prod_hat - rho_hat * term_lagged_prod_expectation_hat

# 4. Calcular contribuições individuais para os momentos m_it = u_jt_hat * Z_it
m_t = [u_jt_hat * k_jt, u_jt_hat * l_jt_minus_1]
m1_t_hat = u_jt_hat * k_jt
m2_t_hat = u_jt_hat * l_jt_minus_1

# 5. Calcular Omega_hat (matriz de variância-covariância dos momentos m_t)
# Omega_hat[i,j] = mean(m_it * m_jt)
Omega_hat_11 = np.mean(m1_t_hat**2)
Omega_hat_12 = np.mean(m1_t_hat * m2_t_hat) # = mean(m2_t_hat * m1_t_hat)
Omega_hat_22 = np.mean(m2_t_hat**2)
Omega_hat = np.array([[Omega_hat_11, Omega_hat_12], [Omega_hat_12, Omega_hat_22]])

# 6. Calcular a matriz de variância-covariância dos estimadores GMM
# Para GMM exatamente identificado (W=I), V_cov(beta_hat) = (1/N) * (G_hat' * G_hat)^{-1} * (G_hat' * Omega_hat * G_hat) * (G_hat' * G_hat)^{-1}
# Ou, de forma mais simples para o caso exatamente identificado: V_cov(beta_hat) = (1/N) * (G_hat^{-1} * Omega_hat * G_hat^{-1})

G_hat_inv = np.linalg.inv(G_hat)
V_cov_beta_hat_no_N = G_hat_inv @ Omega_hat @ G_hat_inv.T # (G_hat')^{-1} = (G_hat)^{-1}'
V_cov_beta_hat = (1/N_gmm) * V_cov_beta_hat_no_N

# 7. Hipótese NuLa H0: beta_k = 0, beta_l = 0
beta_gmm_estimated = np.array([beta_k_gmm, beta_l_gmm])

# 8. Estatística de Wald (Chi-quadrado)
# W = beta_hat' * [V_cov(beta_hat)]^{-1} * beta_hat
V_cov_beta_hat_inv = np.linalg.inv(V_cov_beta_hat)
W_stat = beta_gmm_estimated.T @ V_cov_beta_hat_inv @ beta_gmm_estimated

# 9. Estatística F de Wald
F_stat = W_stat / q_restrictions

# 10. P-valor para a estatística F
# Graus de Liberdade: q_restrictions (numerador), N_gmm - K_params (denominador)
p_value_F = 1 - f_dist.cdf(F_stat, q_restrictions, N_gmm - K_params)

print("\nTeste de Wald para significância conjunta de beta_k e beta_l (H0: beta_k=0, beta_l=0):")
print(f"Número de observações (GMM): {N_gmm}")
print(f"Matriz G_hat: \n{G_hat}")
print(f"Matriz Omega_hat: \n{Omega_hat}")
print(f"Matriz de Variância-Covariância (V_cov_beta_hat): \n{V_cov_beta_hat}")
print(f"Estatística de Wald (Chi^2(q_restrictions)): {W_stat:.4f}")
print(f"Estatística F (q_restrictions): {N_gmm - K_params}: {F_stat:.4f}")
print(f"P-valor (F): {p_value_F:.4f}")

Teste de Wald para significância conjunta de beta_k e beta_l (H0: beta_k=0, beta_l=0):
Número de observações (GMM): 2047
Matriz G_hat:
[[-7.68205934 -1.58854136]
 [-1.89250106 -0.34947153]]
Matriz Omega_hat:
[[59.6826729 14.58022482]
 [14.58022482 3.83646026]]
Matriz de Variância-Covariância (V_cov_beta_hat):
[[ 0.0036915 -0.01564909]
 [-0.01564909 0.0765797 ]]
Estatística de Wald (Chi^2(2)): 215.9948
Estatística F(2, 2045): 107.9974
P-valor (F): 0.0000

3. Calcule o erro-padrão robusto:

SE_{\hat{\theta}} = \sqrt{\frac{1}{n} \hat{Avar}(\hat{\theta}(\hat{W}))}
```

```
In [5]: from scipy.stats import norm

# Os erros padrão robustos são a raiz quadrada da matriz de Var-Cov
se_beta_k = np.sqrt(V_cov_beta_hat[0, 0])
se_beta_l = np.sqrt(V_cov_beta_hat[1, 1])

print("\nErros Padrão Robustos:")
print(f"beta_k: Coef = {se_beta_k:.4f}")
print(f"beta_l: Coef = {se_beta_l:.4f}")

# Estatísticas t e p-valores para significância individual
t_beta_k = beta_k_gmm / se_beta_k
t_beta_l = beta_l_gmm / se_beta_l

# P-valores (usando distribuição normal padrão para grandes amostras)
p_value_beta_k = 2 * (1 - norm.cdf(np.abs(t_beta_k)))
p_value_beta_l = 2 * (1 - norm.cdf(np.abs(t_beta_l)))

print("\nTestes de Significância Individual (usando erros padrão robustos):")
print(f"beta_k: Coef = {beta_k_gmm:.4f}, SE = {se_beta_k:.2f}, p-valor = {p_value_beta_k:.4f}")
print(f"beta_l: Coef = {beta_l_gmm:.4f}, SE = {se_beta_l:.2f}, t = {t_beta_l:.2f}, p-valor = {p_value_beta_l:.4f}")

Erros Padrão Robustos:
SE(beta_k): 0.0608
SE(beta_l): 0.2767

Testes de Significância Individual (usando erros padrão robustos):
beta_k: Coef = 0.3618, SE = 0.0608, t = 5.95, p-valor = 0.0000
beta_l: Coef = -0.1739, SE = 0.2767, t = -0.63, p-valor = 0.5297
```

4. Calcule as variáveis de transparência de Andrews, Gentzkow e Shapiro (2017, 2020). A medida de sensibilidade (AGS 2017):

$$\hat{\Lambda} = -(\hat{G}(\hat{\theta})' \hat{W} \hat{G}(\hat{\theta}))^{-1} \hat{G}(\hat{\theta})' \hat{W}$$

```
In [11]: G_hat_inv = np.linalg.inv(G_hat)
Lambda_hat = -G_hat_inv
print(f"Matriz Lambda_hat: \n{Lambda_hat}")

print("\nInterpretação:")
print("Cada linha corresponde a um parâmetro (beta_k, beta_l).")
print("Cada coluna corresponde a uma condição de momento (E[u*k]=0, E[u*l_lag]=0).")
print("Exemplo: Lambda[0, 0] = Lambda_hat[0, 0]:.4f indica que um aumento de 1 unidade")
print("no valor do primeiro momento (relativo ao capital) mudaria a estimativa de beta_k")
print("em {Lambda_hat[0, 0]:.4f} unidades.")

Matriz Lambda_hat:
[[-1.08647887 4.93864725]
 [ 5.88633349 -23.88290421]]

Interpretação:
- Cada linha corresponde a um parâmetro (beta_k, beta_l).
- Cada coluna corresponde a uma condição de momento (E[u*k]=0, E[u*l_lag]=0).
- Exemplo: Lambda[0, 0] = -1.0865 indica que um aumento de 1 unidade no valor do primeiro momento (relativo ao capital) mudaria a estimativa de beta_k em -1.0865 unidades.
```